BFS and DFS

1.BFS: The program builds an undirected graph from user input, runs Breadth-First Search (BFS) from a start node to a goal node, and animates the search with colors. Here is how it works, section by section.

```
# ----------------------------------
# Install libraries (run once)
# ----------------------------------
!pip install networkx matplotlib

# ----------------------------------
# Import libraries
# ----------------------------------
import networkx as nx
import matplotlib.pyplot as plt
from collections import deque

# ----------------------------------
# Take graph input from user
# ----------------------------------
G = nx.Graph()

n = int(input("Enter number of edges: "))

print("Enter edges (node1 node2):")
for _ in range(n):
    u, v = input().split()
    G.add_edge(u, v)

start = input("Enter start node: ")
goal = input("Enter goal node: ")

pos = nx.spring_layout(G)

# ----------------------------------
# Draw function
# ----------------------------------
def draw_graph(visited, current):
    plt.clf()

    node_colors = []
    for node in G.nodes():
        if node == current:
            node_colors.append('red')    # current node
        elif node in visited:
            node_colors.append('green') # visited
        else:
            node_colors.append('lightblue')

    nx.draw(G, pos, with_labels=True, node_color=node_colors,
node_size=3000)
    plt.title(f"BFS Visiting: {current}")
```

```
    plt.pause(1)

# ------------------------------------
# BFS Algorithm
# ------------------------------------
def bfs():
    OPEN = deque([start])
    visited = set([start])
    CLOSE = []

    plt.figure()

    while OPEN:
        N = OPEN.popleft()
        CLOSE.append(N)

        draw_graph(CLOSE, N)

        if N == goal:
            print("\nGoal Found!")
            print("Traversal:", CLOSE)
            break

        for neighbor in G[N]:
            if neighbor not in visited:
                OPEN.append(neighbor)
                visited.add(neighbor)

    plt.show()

# Run BFS
bfs()
```

Explanation :

1. Installing and importing libraries

```python
!pip install networkx matplotlib
```

The `!` runs a shell command from a notebook cell. It installs two packages:

- **networkx** builds and manages graphs (nodes, edges, neighbors).
- **matplotlib** draws the graph.

```python
import networkx as nx
import matplotlib.pyplot as plt
from collections import deque
```

- `deque` is a double-ended queue. BFS needs a FIFO queue, and `deque.popleft()` is O(1), whereas `list.pop(0)` is O(n).

2. Taking graph input

```python
G = nx.Graph()
n = int(input("Enter number of edges: "))
for _ in range(n):
    u, v = input().split()
    G.add_edge(u, v)
```

- `nx.Graph()` creates an **undirected** graph, so an edge `A B` means you can travel A→B and B→A.
- You enter the number of edges, then one edge per line, e.g. `A B`.
- `add_edge` creates the nodes automatically if they don't exist yet.
- Nodes are stored as **strings**, which is why start and goal are also read as strings.

```python
start = input("Enter start node: ")
goal = input("Enter goal node: ")
pos = nx.spring_layout(G)
```

- `spring_layout` computes fixed (x, y) coordinates for each node. It is computed **once** so the nodes don't jump around between animation frames.

Example input:

```
5
A B
A C
B D
C D
D E
Start: A   Goal: E
```

```
Start: A   Goal: E
```

3. The `draw_graph` function

```python
def draw_graph(visited, current):
    plt.clf()
```

Clears the figure so each frame is drawn fresh.

```python
node_colors = []
for node in G.nodes():
    if node == current:        node_colors.append('red')
    elif node in visited:      node_colors.append('green')
    else:                      node_colors.append('lightblue')
```

Colors follow this scheme:

Color	Meaning
Red	The node being processed right now
Green	Already processed (in `CLOSE`)
Light blue	Not processed yet

The order of the checks matters: `current` is also in `visited`, so it must be tested first or it would show as green.

```python
nx.draw(G, pos, with_labels=True, node_color=node_colors, node_size=3000)
plt.title(f"BFS Visiting: {current}")
plt.pause(1)
```

- The colors list must be in the same order as `G.nodes()`, which is why it is built by iterating over it.
- `plt.pause(1)` redraws the canvas and waits 1 second, creating the animation effect.

4. The BFS algorithm

```python
OPEN = deque([start])
visited = set([start])
CLOSE = []
```

Three data structures are used:

- **OPEN**: the queue of discovered nodes waiting to be processed (the "frontier").
- **visited**: a set of nodes that have ever been *added* to OPEN. This prevents adding a node twice, with O(1) lookups.
- **CLOSE**: the list of nodes already *processed*, in order. This is the traversal order.

```python
while OPEN:
    N = OPEN.popleft()
    CLOSE.append(N)
    draw_graph(CLOSE, N)
```

Each iteration takes the **oldest** node from the queue, which is what makes the search level by level. It marks that node as processed and draws it.

```python
    if N == goal:
        print("\nGoal Found!")
        print("Traversal:", CLOSE)
        break
```

If the node just taken is the goal, the search stops and prints the order in which nodes were visited.

If the node just taken is the goal, the search stops and prints the order in which nodes were visited.

```python
    for neighbor in G[N]:
        if neighbor not in visited:
            OPEN.append(neighbor)
            visited.add(neighbor)
```

`G[N]` gives the neighbors of `N`. Each unseen neighbor is added to the back of the queue and marked visited immediately, so it cannot be queued again through another path.

```python
plt.show()
```

Keeps the final frame displayed.

5. Dry run on the example

Edges: A–B, A–C, B–D, C–D, D–E. Start = A, Goal = E.

Step	Node processed (N)	OPEN after	CLOSE
1	A	[B, C]	[A]
2	B	[C, D]	[A, B]
3	C	[D] (D already visited)	[A, B, C]
4	D	[E]	[A, B, C, D]
5	E → goal found	[]	[A, B, C, D, E]

Output: `Traversal: ['A', 'B', 'C', 'D', 'E']`

2. DFS Algorithm

Code:

```
# ----------------------------------
# Install libraries (run once)
# ----------------------------------
!pip install networkx matplotlib

# ----------------------------------
# Import libraries
# ----------------------------------
import networkx as nx
import matplotlib.pyplot as plt

# ----------------------------------
# Take graph input from user
# ----------------------------------
G = nx.Graph()

n = int(input("Enter number of edges: "))

print("Enter edges (node1 node2):")
for _ in range(n):
    u, v = input().split()
    G.add_edge(u, v)

start = input("Enter start node: ")
goal = input("Enter goal node: ")

pos = nx.spring_layout(G)

# ----------------------------------
# Draw function
# ----------------------------------
def draw_graph(visited, current):
    plt.clf()

    node_colors = []
    for node in G.nodes():
        if node == current:
            node_colors.append('red')    # current node
        elif node in visited:
            node_colors.append('green') # visited
        else:
            node_colors.append('lightblue')

    nx.draw(G, pos, with_labels=True, node_color=node_colors,
node_size=3000)
    plt.title(f"DFS Visiting: {current}")
    plt.pause(1)
```

```
# ------------------------------------
# DFS Algorithm
# ------------------------------------
def dfs():
    OPEN = [start]
    visited = set([start])
    CLOSE = []

    plt.figure()

    while OPEN:
        N = OPEN.pop()
        CLOSE.append(N)

        draw_graph(CLOSE, N)

        if N == goal:
            print("\nGoal Found!")
            print("Traversal:", CLOSE)
            break

        for neighbor in G[N]:
            if neighbor not in visited:
                OPEN.append(neighbor)
                visited.add(neighbor)

    plt.show()

# Run DFS
dfs()
```

Explanation:

This program is almost identical to the BFS one from before. It builds an undirected graph from user input, then searches from a start node to a goal node using **Depth-First Search (DFS)**, animating each step with colors. The only real difference is how the OPEN list is used: as a **stack** instead of a queue.

1. Installing and importing libraries

```python
!pip install networkx matplotlib
```

The `!` runs a shell command from a notebook cell. It installs two packages:

- **networkx** builds and manages graphs (nodes, edges, neighbors).
- **matplotlib** draws the graph.

```python
import networkx as nx
import matplotlib.pyplot as plt
from collections import deque
```

- `deque` is a double-ended queue. BFS needs a FIFO queue, and `deque.popleft()` is O(1), whereas `list.pop(0)` is O(n).

2. Taking graph input

```python
G = nx.Graph()
n = int(input("Enter number of edges: "))
for _ in range(n):
    u, v = input().split()
    G.add_edge(u, v)
```

- `nx.Graph()` creates an **undirected** graph, so an edge `A B` means you can travel A→B and B→A.
- You enter the number of edges, then one edge per line, e.g. `A B`.
- `add_edge` creates the nodes automatically if they don't exist yet.
- Nodes are stored as **strings**, which is why start and goal are also read as strings.

```python
start = input("Enter start node: ")
goal = input("Enter goal node: ")
pos = nx.spring_layout(G)
```

- `spring_layout` computes fixed (x, y) coordinates for each node. It is computed **once** so the nodes don't jump around between animation frames.

Example input:

```
5
A B
A C
B D
C D
D E
Start: A   Goal: E
```

3. The `draw_graph` function

```python
def draw_graph(visited, current):
    plt.clf()
```

Clears the figure so each frame is drawn fresh.

```python
node_colors = []
for node in G.nodes():
    if node == current:        node_colors.append('red')
    elif node in visited:      node_colors.append('green')
    else:                      node_colors.append('lightblue')
```

Colors follow this scheme:

Color	Meaning
🔴 Red	The node being processed right now
🟢 Green	Already processed (in `CLOSE`)
🔵 Light blue	Not processed yet

The order of the checks matters: `current` is also in `visited`, so it must be tested first or it would show as green.

```python
nx.draw(G, pos, with_labels=True, node_color=node_colors, node_size=3000)
plt.title(f"BFS Visiting: {current}")
plt.pause(1)
```

- The colors list must be in the same order as `G.nodes()`, which is why it is built by iterating over it.
- `plt.pause(1)` redraws the canvas and waits 1 second, creating the animation effect.

---

4. The BFS algorithm

```python
OPEN = deque([start])
visited = set([start])
CLOSE = []
```

Three data structures are used:

- **OPEN**: the queue of discovered nodes waiting to be processed (the "frontier").
- **visited**: a set of nodes that have ever been *added* to OPEN. This prevents adding a node twice, with O(1) lookups.
- **CLOSE**: the list of nodes already *processed*, in order. This is the traversal order.

```python
while OPEN:
    N = OPEN.popleft()
    CLOSE.append(N)
    draw_graph(CLOSE, N)
```

Each iteration takes the **oldest** node from the queue, which is what makes the search level by level. It marks that node as processed and draws it.

```python
    if N == goal:
        print("\nGoal Found!")
        print("Traversal:", CLOSE)
        break
```

If the node just taken is the goal, the search stops and prints the order in which nodes were visited.

```python
    for neighbor in G[N]:
        if neighbor not in visited:
            OPEN.append(neighbor)
            visited.add(neighbor)
```

`G[N]` gives the neighbors of `N`. Each unseen neighbor is added to the back of the queue and marked visited immediately, so it cannot be queued again through another path.

```python
plt.show()
```

Keeps the final frame displayed.

5. Dry run on the example

Edges: A–B, A–C, B–D, C–D, D–E. Start = A, Goal = E.

Step	Node processed (N)	OPEN after	CLOSE
1	A	[B, C]	[A]
2	B	[C, D]	[A, B]
3	C	[D] (D already visited)	[A, B, C]
4	D	[E]	[A, B, C, D]
5	E → goal found	[]	[A, B, C, D, E]

Output: `Traversal: ['A', 'B', 'C', 'D', 'E']`